



DinisCruz minor formatting fix

1d3aa11 · 12 minutes ago🕒

373 lines (249 loc) · 21.2 KB

PreviewCodeBlame

📄

🔗

Raw

📄

⬇

✎

⌵

On Fractal Scopes: The Recursive Architecture of Issues-FS

Document: on-fractal-scopes
Version: v0.2.3
Date: 2026-02-18
Status: Active
Author: Librarian (compiled from 35+ sources across the ecosystem)
Audience: All roles, new contributors, external stakeholders

Purpose

This is the standalone reference document for the **fractal principle** in Issues-FS -- the idea that the same structural pattern (nodes with edges, meaning through connectivity) repeats at every level of zoom, from the root Lexicon down to a single checkbox inside a task.

The concept appears in over 35 files across the ecosystem but has never had its own dedicated document. This fills that gap. It draws on the canonical treatment in *Thinking in Graphs* (Part 3), the formal model in *Lexicon Architecture v2*, the filesystem nesting in the implementation briefs, and the stakeholder's vision as expressed in briefs and interviews.

Source Documents

Document	Location	What it contributes
Thinking in Graphs	modules/Issues-FS__Docs/docs/to_classify/v0_4_0__issues-fs__thinking-in-graphs.md	Part 3: The Fractal Principle; Part 6: Graphs of Graphs
Lexicon Architecture v2	modules/Issues-FS__Docs/docs/to_classify/v0_4_0__issues-fs__lexicon-architecture-v2.md	"How: The Fractal Model"; scope resolution; pattern__fractal_scope
Semantic Text Architecture	modules/Issues-FS__Docs/docs/to_classify/6-feb/v0_4_0__issues-fs__semantic-text-architecture.md	Part 2: The Fractal Document Model
Use Case Pattern	modules/Issues-FS__Docs/docs/to_classify/6-feb/v0_4_0__issues-fs__use-case-pattern.md	Use cases as fractal scopes
File Structure Refactoring	humans/Issues-FS__Dev__Human__Dinis_Cruz/briefs/02/10/v0.1.2__briefs__issues-fs-file-structure-refactoring.md	Code must reflect the fractal design
Git/GitHub Comparison	humans/Issues-FS__Dev__Human__Dinis_Cruz/briefs/02/10/v0.1.3__briefs__issues-fs-and-git-github-comparison.md	Fractal vs Git's DAG; recursive nesting as differentiator

Document	Location	What it contributes
Stakeholder Interview	roles/Issues-FS__Dev__Role__Journalist/publications/_interviews/2026-02-10/v0.1.3__chatgtp-stakeholder__raw-interview.md	Vision in the stakeholder's own words
Cartographer Role	roles/Issues-FS__Dev__Role__Cartographer/ROLE.md	Maps as fractal structures
Manual Issues Workflow	modules/Issues-FS__Docs/docs/issues_fs/llm-briefs/v0.2.32__llm-brief__gitgraph-manual-issues-workflow.md	Filesystem nesting: .issues/issues/.../issues/
Librarian Side Capture	modules/Issues-FS__Docs/docs/to_classify/6-feb/v0_4_0__issues-fs__librarian-role-side-capture.md	"A document IS a fractal scope"

Part 1: What "Fractal" Means in Issues-FS

The Core Idea

Issues-FS is not one graph. It is **graphs of graphs of graphs**, at every level of scope:

```
Issues-FS__Lexicon (root)
├── Issues-FS__Service (project)
│   ├── Project-6 (project instance)
│   │   ├── Epic-14 (epic)
│   │   │   ├── Sprint-3 (sprint)
│   │   │   │   ├── Task-23 (task)
│   │   │   │   │   ├── Sub-Task-1 (sub-task)
│   │   │   │   │   │   └── Checklist-Item-3 (checklist)
```

Every level in this hierarchy can introduce its own **nodes, edges, types, and definitions**. The structural pattern -- nodes with edges, meaning through connectivity -- repeats identically at every level. This is what we call the fractal principle.

A project can define issue types that only make sense within that project. An epic can further specialise those types. A single task can define its own sub-vocabulary: checkboxes, acceptance criteria, sub-steps. At no level does a child scope need to "register" its vocabulary with a parent scope.

Why "Fractal" and Not Just "Hierarchical"

Hierarchies are top-down. A child element conforms to its parent's schema, or it is rejected. Traditional issue trackers work this way: Jira defines a global set of issue types and workflows; every project inherits them.

Fractal structures are **self-similar at every scale**. Each scope is a complete, self-contained graph that happens to nest inside a larger graph. The child scope does not inherit from the parent. It can link to the parent's definitions (gaining interoperability) or not (remaining locally meaningful but opaque to cross-scope analysis). Neither choice is wrong.

This distinction matters because it determines how the system handles the real world: partial information, competing definitions, local conventions, and evolving vocabulary. A hierarchical system forces conformity. A fractal system surfaces the differences and lets humans or agents decide what to do about them.

In Dinis Cruz's Words

From the stakeholder interview (10 February 2026):

"What we're doing now is a **fractal way of capturing information**. It's already quite mature structurally -- now we need to implement it.

What's interesting is that this is bigger than issues. It becomes a way to communicate with agents, to communicate with information, to capture documents, structures, and context. **Every level -- repo, folder, issue -- can define its own ontology and taxonomy**. That's the power of the fractal design."

Part 2: Where Fractal Structure Appears

The fractal principle is not an abstract philosophy. It manifests concretely in five distinct layers of the system.

Layer 1: The Filesystem

Issues nest inside issues via `issues/` subdirectories. This is the most visible expression of the fractal principle:

```
.issues/
  issue.json
  issues/
    Project-1/
      issue.json
      issues/
        Phase-1/
          issue.json
          issues/
            Release-4/
              issue.json
              issues/
                Feature-12/
                  issue.json
                  issues/
                    Task-42/
                      issue.json
```

← Root (GitRepo)
← Project
← Phase
← Release
← Feature
← Task

Each `issues/` directory is a complete Issues-FS database at its level. Each level can have its own `config/` directory with `node-types.json` and `link-types.json`. The recursive discovery system (`nodes_list_all()`) walks this tree to find all issues at any depth.

This is a key differentiator from Git. As the Git/GitHub comparison brief notes:

"**Fractal structure** -- Issues inside issues inside issues, recursively. Git doesn't have native nesting of repos (submodules are bolted on and painful)."

Layer 2: The Vocabulary (Scope Resolution)

Every scope can define its own node types, edge types, and vocabulary. When the system encounters a concept, it resolves meaning by walking the scope chain:

```
Task-23 (in Sprint-3, in Project-6)
  type → "Task"

Scope resolution:
1. Does Sprint-3 define "Task"? → No
2. Does Project-6 define "Task"? → Yes: local definition
   └─ links_to → Lexicon:anchor__task (anchor node)

Result: Task-23's type resolves to Project-6's local definition,
which links to the Lexicon anchor. Confidence: high.
```

If resolution reaches the root Lexicon without finding a definition, the system reports honestly: "No definition of X exists in any reachable scope." This is not an error. It is an observation.

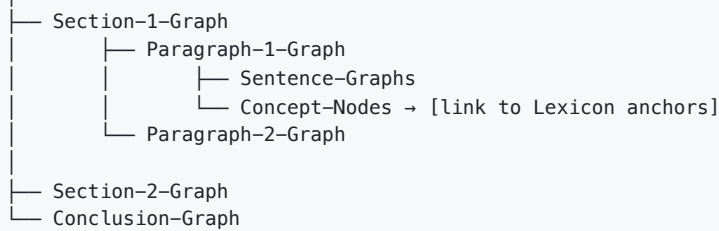
Name clashes are normal. `Task-23` in `Project-6` and `Task-23` in `User-Story-12` are different nodes. Resolution is structural (through graph traversal), not nominal (through label matching).

Divergent definitions are expected. Two projects can define `Review` differently -- one for code review, one for client deliverables. Both can link to the same Lexicon anchor. The analysis tools compute overlap, divergence, and compatibility without forcing conformity.

Layer 3: Documents (The Fractal Document Model)

The Semantic Text Architecture extends the fractal principle to document structure:

Document-Graph (overview)



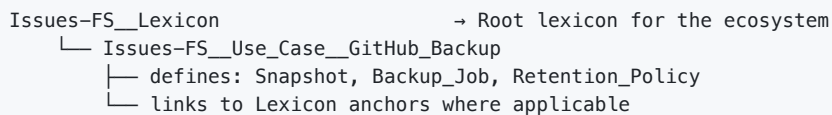
Each level is a complete Issues-FS database (can be stored as its own zip/database). The levels link to each other through explicit edges. This enables:

- **Independent storage:** Load a paragraph graph without loading the entire document
- **Independent versioning:** Section-2 can change without affecting Section-1
- **Composability:** Combine paragraph graphs from different documents into a new analysis
- **Bidirectional traversal:** Zoom in (expand) or zoom out (reduce/summarise)

The Librarian side-capture makes this explicit: **"A document IS a fractal scope."** Each document can define local concepts, and those concepts can link to Lexicon anchors. The document's "ontology" is just its local subgraph.

Layer 4: Use Cases (Fractal Scopes as Products)

Every use-case repository is a fractal scope with its own vocabulary:



Use cases are not "scripts that use Issues-FS." They are **full participants in the graph ecosystem**, able to:

- Define domain-specific node and edge types
- Link to Lexicon anchors for cross-scope interoperability
- Contribute vocabulary back to the Lexicon if concepts prove broadly useful

This is what makes the use-case pattern scalable. Each use case adds to the ecosystem's vocabulary without requiring coordination with other use cases.

Layer 5: Maps and Roles (Fractal Composition)

The Cartographer role applies the fractal principle to maps:

"Maps inherit all the properties of graphs: fractal scoping, anchor nodes, connectivity analysis, honest uncertainty."

Every component on a map can itself be a map:



This is maps of maps of maps -- and since maps are graphs, it is graphs of graphs of graphs. The Cartographer's doctrine assessment is itself fractal: it can be applied to the ecosystem as a whole, to a specific project, to a role, or to a single sprint.

Part 3: The Formal Pattern

The Lexicon Architecture v2 formalises the fractal principle as a named pattern:

pattern__fractal_scope

Definition: A thing that can define its own vocabulary.

Base class: `Base__Scope.py`

Design invariant (L7): "The Lexicon is the root scope, not the only scope. Every repo, project, epic, and issue can define its own vocabulary."

Properties of a Fractal Scope

1. **Self-contained:** A scope is a complete graph. It does not require any parent scope to be meaningful.
2. **Composable:** Scopes nest inside other scopes. A child scope's graph is reachable from the parent's graph through traversal.
3. **Locally definable:** A scope can introduce nodes, edges, and types that exist only within that scope. No registration with a parent scope is required.
4. **Optionally connected:** A scope may link its definitions to anchor nodes in a parent scope (gaining interoperability) or not (remaining locally meaningful).
5. **Independently storable:** Each scope can be stored as its own Issues-FS database, zip file, or Memory-FS instance.
6. **Independently versionable:** A scope can change without affecting sibling or parent scopes.

Every Scope Is a Potential Lexicon

The fractal principle means that any scope can serve as a "lexicon" for its children:

```
Issues-FS__Lexicon      → Root lexicon for the ecosystem
├─ Issues-FS__Service__GitHub → Lexicon for GitHub integration concepts
│   └─ defines: PR_Review, Merge_Conflict, Label_Sync
│   └─ links to Lexicon anchors where applicable
│       └─ Project-6      → Lexicon for this project's concepts
│           └─ defines: Sprint_Goal, Retrospective, Team_Velocity
│           └─ some link to Lexicon, some are local-only
│               └─ Epic-14 → Lexicon for this epic's concepts
│                   └─ defines: Spike, Tech_Debt_Item
│                   └─ local-only definitions
```

Each level adds nodes and edges. Each level can link upward -- or not. The richness of cross-scope connectivity is a choice, not a requirement. The analysis tools report the consequences of that choice: "Your scope has 5 local definitions. 3 link to Lexicon anchors (high interoperability). 2 are local-only (opaque to cross-scope analysis)."

Part 4: Why This Matters

Incompatibility and Missing Pieces Are Expected

A fractal system with distributed vocabulary will inevitably produce:

- **Gaps:** An issue references a type that doesn't have a definition node anywhere in its reachable graph. The system reports: "This issue claims to be of type X, but no definition of X is reachable from this scope."
- **Partial mappings:** An issue type in one scope maps to 3 out of 5 aspects of a reference definition. The system reports: "This type partially matches the reference. Here's what matches and what doesn't."
- **Contradictions:** Two definitions in overlapping scopes make conflicting claims. The system reports: "Definition A says status 'closed' is terminal. Definition B says 'closed' can transition to 'reopened.' These are in conflict within scope X."

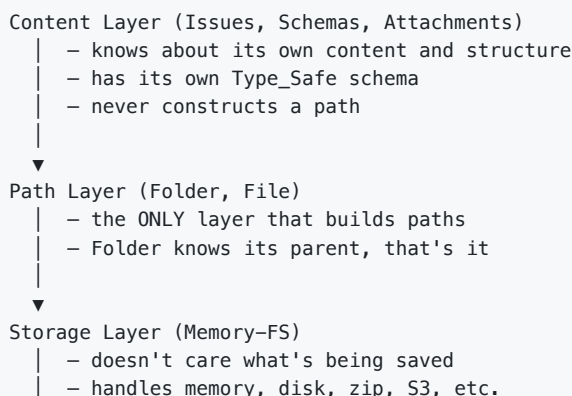
None of these are errors. They are observations that the graph analysis surfaces for human or agent judgment. The system operates with partial, uncertain, and sometimes contradictory information -- just like the real world.

The Code Must Reflect the Design

The P1 file structure refactoring brief (10 February 2026) makes this point forcefully:

"The whole philosophy of Issues FS is thinking in graphs, thinking in independent and fractal structures. But the source code doesn't materialise that."

The refactoring separates three layers that correspond to the fractal architecture:



Core primitives should not know about each other. A core primitive should just BE. It should be given content and asked to save. It should not know where it lives, what is next to it, or how the folder structure works. This is how you prevent path injection: the things that handle user content never touch paths, and the things that handle paths never interpret user content.

Beyond Issue Tracking

The fractal design makes Issues-FS more than an issue tracker. As the stakeholder interview and multiple briefs articulate, it becomes a general-purpose system for capturing structured information:

- **Risk registers** become fractal: a risk contains vulnerabilities, which contain attack vectors, which contain mitigations -- each level its own graph.
- **RSS feeds** become fractal: a feed contains entries, which contain paragraphs, which contain concepts -- each linked to domain ontologies.
- **Communication** becomes fractal: a board presentation contains persona-translated sections, which contain claims, which link back to evidence at arbitrary depth.
- **Ontologies and taxonomies** are fractal: an O&T for risk descriptions can extend a base O&T, which itself extends a root vocabulary.

The fractal principle is what makes all of this possible with a single architecture. The same structural pattern -- nodes with edges, meaning through connectivity, scope resolution through traversal -- applies whether you are tracking bugs or modelling geopolitical risk.

Part 5: How It Compares to Other Approaches

vs. Traditional Issue Trackers (Jira, GitHub Issues)

Traditional issue trackers use a **flat or two-level hierarchy** (project → issues, with optional sub-tasks). Issue types and workflows are defined globally or per project. There is no recursive nesting, no local vocabulary, no scope resolution chain.

Issues-FS supports arbitrary depth nesting with local vocabulary at every level. A team can start with flat issues and grow to deep hierarchies as complexity demands, without changing the underlying architecture.

vs. Git's DAG

Git's data model is a DAG (directed acyclic graph) of commits, trees, and blobs. It is append-only and linear. Submodules exist but are "bolted on and painful."

Issues-FS is a full graph -- nodes and edges in any direction, cycles allowed, arbitrary relationship types -- with native recursive nesting built into the filesystem model.

vs. Schema.org / Semantic Web

The Semantic Web attaches meaning *to nodes* (a node IS a `schema:Review`). Issues-FS derives meaning *from edges* (a node has edges that are `similar_to` components of `schema:Review`). The fractal principle means every scope can define its own vocabulary and optionally link to external references, rather than conforming to a global ontology.

vs. Fixed-Level Architecture Frameworks (C4)

The C4 model defines four fixed levels: System, Container, Component, Code. As the *Journey* document notes:

"Reality is fractal -- a 'Container' is itself a 'System' when you zoom in. A 'Component' might be a library with its own architecture. The 4 is arbitrary."

Issues-FS does not impose a fixed number of levels. Levels emerge from traversal, not from pre-defined categories.

Part 6: Implementation Status

What Exists Today

Feature	Status	Location
Filesystem nesting (<code>issues/issues/.../issues/</code>)	Implemented	Core library
Recursive node discovery (<code>nodes_list_all()</code>)	Implemented (B10)	<code>modules/Issues-FS/</code>
Per-scope <code>node-types.json</code> and <code>link-types.json</code>	Implemented	<code>.issues/config/</code>
Cross-graph edges (cross-database references)	Design only	Thinking in Graphs, Part 6
Scope resolution (walk the chain to find definitions)	Design only	Lexicon Architecture v2
<code>Base__Scope.py</code> / <code>pattern__fractal_scope</code>	Design only	Lexicon Architecture v2
Fractal document model	Design only	Semantic Text Architecture
Analysis tools (confidence, compatibility, gaps)	Design only	Thinking in Graphs, Part 5

What Needs Building

1. **Scope resolution engine** -- Walk the scope chain to resolve types and definitions. Report gaps, partial mappings, and contradictions.
2. **Cross-graph edge support** -- Enable edges that cross database/scope boundaries. This is the mechanism that makes fractal scopes interoperable.
3. **Base__Scope implementation** -- The formal pattern class that encapsulates what it means to be a fractal scope: local vocabulary, optional upward linking, independent storage.
4. **Analysis tooling** -- Confidence scoring, compatibility computation, gap detection, conflict surfacing -- all computed from graph traversal, not from schema validation.
5. **CLI support** -- The `issues-fs scope` command group for managing fractal scopes, listed in the CLI architecture brief but not yet implemented.

Part 7: Summary

The fractal principle is not a feature of Issues-FS. It is **the architecture**. Every design decision -- from filesystem nesting to vocabulary scoping to document structure to use-case packaging -- flows from the same idea: the same structural pattern repeats at every level, each scope is self-contained and self-describing, and interoperability comes from optional edges to shared reference points rather than from mandatory conformity to global schemas.

The five layers where this manifests:

1. **Filesystem** -- `issues/` directories nested recursively
2. **Vocabulary** -- Scope resolution chain from child to root Lexicon
3. **Documents** -- Sections, paragraphs, sentences as nested graphs

4. **Use cases** -- Each repo is a fractal scope with local vocabulary

5. **Maps and roles** -- Components drill down into their own maps/graphs

The system's posture toward the messiness this creates -- gaps, partial mappings, contradictions, name clashes -- is honest observation rather than enforcement. The graph analysis tools report what they find. Humans and agents decide what to do about it.

This is what makes Issues-FS a fractal system rather than merely a hierarchical one. Hierarchies enforce conformity downward. Fractals repeat structure at every scale while preserving autonomy at every level.

*Compiled by the Librarian from 35+ sources across the Issues-FS ecosystem. For the canonical philosophical treatment, see *Thinking in Graphs*, Parts 3 and 6. For the formal technical model, see *Lexicon Architecture v2*, "How: The Fractal Model."*